Deep Learning Lab

Experiment No.: 2

a. Design a single unit perceptron for classification of a linearly separable binary dataset without using pre-defined models. Use the Perceptron () from sklearn.
b. Identify the problem with single unit Perceptron. Classify using Or-, And and Xor data and analyze the result.

Experiment No. 2

1. **Aim:** a. Design a single unit perceptron for classification of a linearly separable binary dataset without using pre-defined models. Use the Perceptron() from sklearn.
   b. Identify the problem with single unit Perceptron. Classify using Or-, And and Xor data and analyze the result.

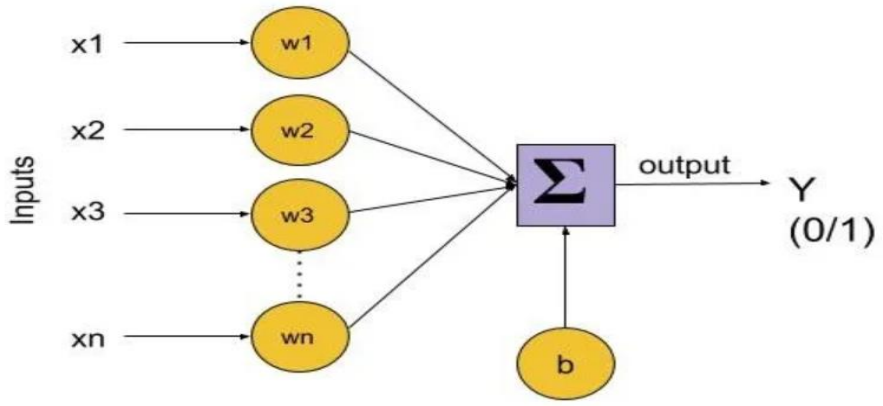
2. **Objectives:** From this experiment, the student will be able to learn the foundation of deep neural networks.

3. **Outcomes:**
   - Students will be to implement basic neural network models of machine learning.

4. **Software Required :** Python

5. Theory:

The perceptron has many inputs(often called features) that are fed into a Linear unit that produces one binary output. Therefore, perceptron's can be applied in solving Binary Classification problems where the sample is to be identified as belonging to one of the predefined two classes.
Since Perceptrons are Binary Classifiers we can define their computation as follows:



Schematic of Perceptron

$$f_\Phi(\mathbf{x}) = \begin{cases} 1 & \text{if } b + \mathbf{w} \cdot \mathbf{x} > 0 \\ 0 & \text{otherwise} \end{cases}$$

Let's recall that the dot product of two vectors of length n ($1 \le i \le n$) is

$$W \cdot X = \sum_i W_i \cdot X_i$$

Steps required for classification using Perceptron:

There are various steps involved in performing classification using the Perceptron algorithm in Scikit-Learn:

1. Data preparation: Preprocess and load your dataset. A training set and a testing set should be separated.
2. Add Required Libraries: Import Scikit-Learn along with the other necessary libraries.
3. Perceptron Model Construction: Set hyperparameters like the learning rate and maximum iterations when creating a Perceptron classifier.
4. Training of the Model: Fit the training set of data to the perceptron model.
5. Make predictions: Based on the testing data, use the trained model to make predictions.
6. Model's performance evaluation: Utilize metrics such as accuracy, precision, recall, and F1-score to evaluate the model's performance.
7. Visualize the outcomes (optional): The decision boundary and the data points can be shown to help you see how the model categorizes cases.
8. A strong Python framework for machine learning called Scikit-Learn offers straightforward and effective tools for modeling and data analysis. Use **pip or conda** to install scikit-learn if you already have a functioning installation of numpy and scipy.

Sklearn (Scikit-Learn):

```
pip install -U scikit-learn
conda install -c conda-forge scikit-learn
```

1. Single-Layer Perceptron

A **single-layer perceptron** is the simplest form of an artificial neural network and consists of:

- **Input Layer**: Takes the input features.
- **Single Neuron**: Applies weights to the inputs, sums them, and passes the result through an activation function (e.g., step function) to produce an output.
- **Output**: Typically used for linearly separable binary classification problems.

Mathematical Representation

$$y = \text{activation}\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

Where:

- w_i: Weight for the i-th input.
- x_i: i-th input feature.
- b: Bias term.
- **Activation Function:** Typically a step function:

$$y = \begin{cases} 1 & \text{if } \sum(w_i x_i + b) \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

2. Multi-Layer Perceptron (MLP)

A **multi-layer perceptron** (MLP) is a more complex version of the perceptron. It consists of:

- **Input Layer**: Takes the input features.
- **Hidden Layers**: One or more layers of neurons between the input and output layers. Each neuron uses an activation function to introduce non-linearity.
- **Output Layer**: Provides the final output.

Key Features of MLP

1. **Universal Approximation**: Can approximate any function with enough hidden neurons.
2. **Non-Linear Activation Functions**: Sigmoid, ReLU, or Tanh allows solving non-linear problems.
3. **Backpropagation**: Used for learning weights and biases through gradient descent.

B. How the Perceptron works with Logic gates (AND, OR, NOT, and so on).

First, we need to know that the Perceptron algorithm states that:

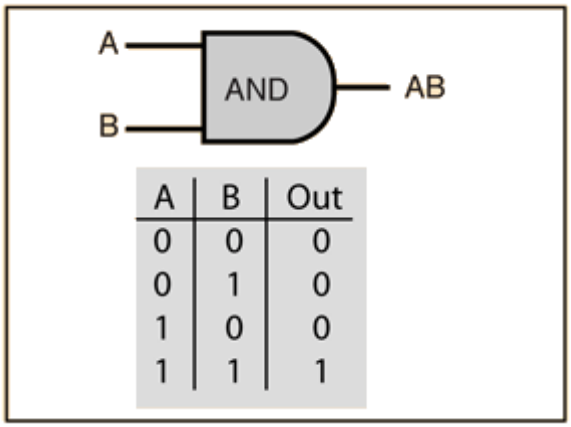
Prediction (y`) = 1 if Wx+b > 0 and 0 if Wx+b ≤ 0
Also, the steps in this method are very similar to how Neural Networks learn, which is as follows;

- Initialize weight values and bias
- Forward Propagate
- Check the error
- Backpropagate and Adjust weights and bias
- Repeat for all training examples

Now that we know the steps, let's get up and running:

AND Gate

From our knowledge of logic gates, we know that an AND logic table is given by the diagram below



A	B	Out
0	0	0
0	1	0
1	0	0
1	1	1

AND Gate

First, we need to understand that the output of an AND gate is 1 only if both inputs (in this case, x1 and x2) are 1. So, following the steps listed above;

Row 1

- From w1*x1+w2*x2+b, initializing w1, w2, as 1 and b as –1, we get; *x1(1)+x2(1)–1*
- Passing the first row of the AND logic table (x1=0, x2=0), we get; *0+0–1 = –1*
- From the Perceptron rule, if Wx+b≤0, then y`=0. Therefore, this row is correct, and no need for Backpropagation.

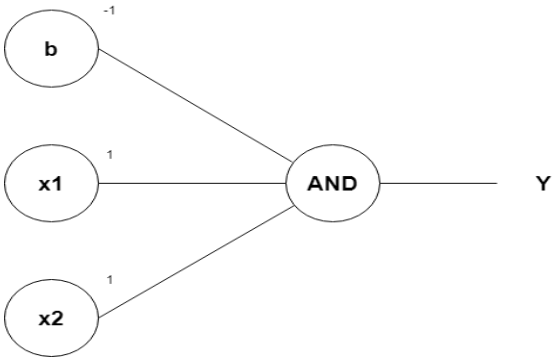
Row 2

- Passing (x1=0 and x2=1), we get;*0+1–1 = 0*
- From the Perceptron rule, if Wx+b≤0, then y`=0. This row is correct, as the output is 0 for the AND gate.
- From the Perceptron rule, this works (for both row 1, row 2 and 3).

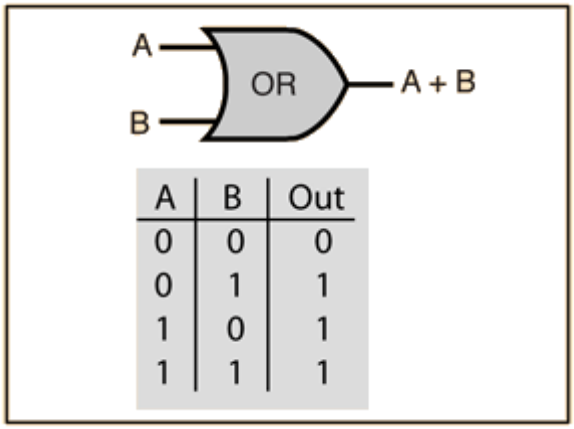
Row 3

- Passing (x1=1 and x2=1), we get; *1+1–1 = 1*
- Again, from the perceptron rule, this is still valid.

Therefore, we can conclude that the model to achieve an AND gate, using the Perceptron algorithm is; *x1+x2–1*.



OR Gate



A	B	Out
0	0	0
0	1	1
1	0	1
1	1	1

OR Gate

From the diagram, the OR gate is 0 only if both inputs are 0.

Row 1

- From w1x1+w2x2+b, initializing w1, w2, as 1 and b as –1, we get;*x1(1)+x2(1)–1*
- Passing the first row of the OR logic table (x1=0, x2=0), we get;*0+0–1 = –1*
- From the Perceptron rule, if Wx+b≤0, then y`=0. Therefore, this row is correct.

Row 2

- Passing (x1=0 and x2=1), we get;*0+1–1 = 0*
- From the Perceptron rule, if Wx+b <= 0, then y`=0. Therefore, this row is incorrect.
- So we want values that will make inputs x1=0 and x2=1 give y` a value of 1. If we change w2 to 2, we have;*0+2–1 = 1*
- From the Perceptron rule, this is correct for both the row 1 and 2.

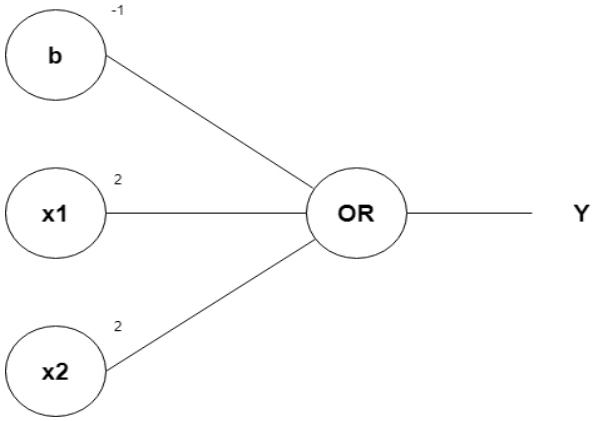
Row 3

- Passing (x1=1 and x2=0), we get;*1+0–1 = 0*
- From the Perceptron rule, if Wx+b <= 0, then y`=0. Therefore, this row is incorrect.
- Since it is similar to that of row 2, we can just change w1 to 2, we have;*2+0–1 = 1*
- From the Perceptron rule, this is correct for both the row 1, 2 and 3.

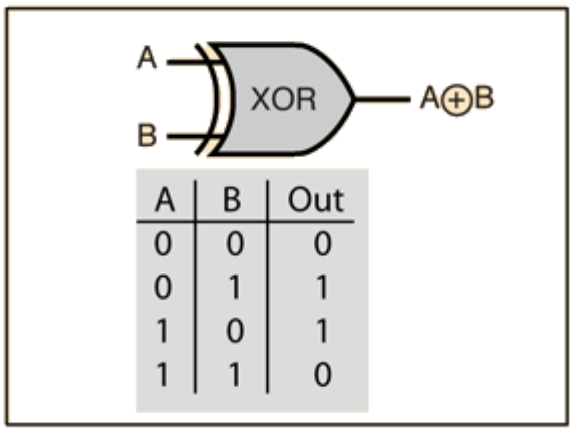
Row 4

- Passing (x1=1 and x2=1), we get;*2+2–1 = 3*

Therefore, we can conclude that the model to achieve an OR gate, using the Perceptron algorithm is;
2x1+2x2–1



XOR Gate

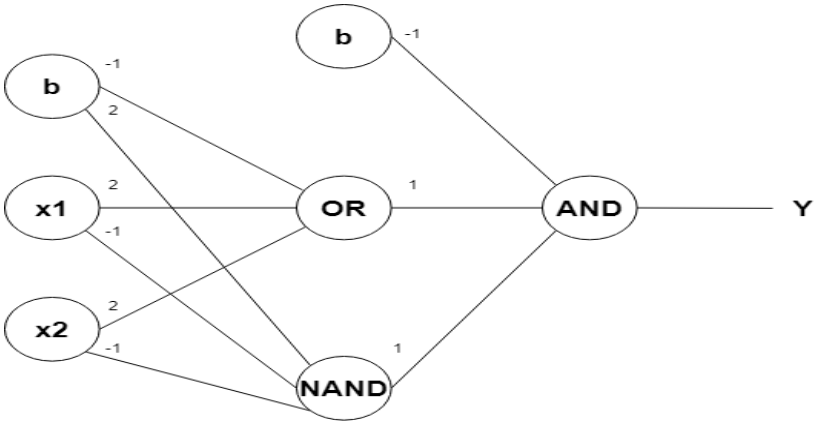


A	B	Out
0	0	0
0	1	1
1	0	1
1	1	0

XOR Gate

The boolean representation of an XOR gate is;
x1x`2 + x`1x2
We first simplify the boolean expression
x`1x2 + x1x`2 + x`1x1 + x`2x2
x1(x`1 + x`2) + x2(x`1 + x`2)
(x1 + x2)(x`1 + x`2)
(x1 + x2)(x1x2)`
From the simplified expression, we can say that the XOR gate consists of an OR gate (x1 + x2), a NAND gate (-x1-x2+1) and an AND gate (x1+x2–1.5).
This means we will have to combine 2 perceptrons:

- OR (2x1+2x2–1)
- NAND (-x1-x2+2)
- AND (x1+x2–1)



6. Conclusion/ Results.

The perceptron is a foundational algorithm in machine learning, offering valuable insights into the principles of linear decision boundaries and binary classification. It is efficient for solving simple, linearly separable problems such as AND and OR gates, making it an excellent starting point for understanding the mechanics of weights, biases, and iterative learning. Despite its simplicity, the perceptron introduces key concepts like weight adjustment and learning rates, which form the basis for more advanced models. However, it has limitations, particularly in handling non-linear problems like XOR, which require more sophisticated approaches.

7. Viva Question

1. what are the weights and bias for the AND perceptron?
2. What is a perceptron and how does it work?
3. Explain the architecture of a single-layer perceptron.
4. What is the role of the activation function in a perceptron?
5. How does the perceptron algorithm learn?
6. What is the purpose of the weights and bias in a single-layer perceptron?
7. What is the significance of the threshold in a perceptron?
8. What is the difference between a single-layer perceptron and a multi-layer perceptron?
9. What are hidden layers in a multi-layer perceptron, and why are they necessary?
10. Explain backpropagation in the context of multi-layer perceptrons.
11. How do multi-layer perceptrons overcome the limitations of single-layer perceptrons?

8. References:

1. Francois Chollet, “Deep learning with Python” – Manning Publications.
2. Ian Goodfellow, Yoshua Bengio, Aaron Courville. ―Deep Learning‖, MIT Press Ltd, 2016